ЛАБОРАТОРНА РОБОТА №4
Дослідження та порівняння різних типів паролів

Мета: Дослідження різних типів паролів, їх стійкості та порівняння відомих методів створення паролів, таких як випадкові символи, фрази та інші, з використанням програм для генерації та аналізу паролів, з метою виявлення сильних та слабких сторін методів захисту паролів.

Завдання: Дослідити та порівняти різні типи паролів на криптостійкість. Для цього розробити програму генерації паролів та їх хешування. Для хешування використати вже вбудовані алгоритми хешування такі як ***MD5*** та аналогічні, підключити стороні бібліотеки, наприклад ***OpenSSl*** та аналогічні. За допомогою інструмента для брутфорс атак на паролі ***John the Ripper*** протестувати міцність згенерованих паролів. Провести аналіз результатів.

Хід роботи

1. Огляд різних типів паролів (наприклад, числові паролі, літеро-цифрові паролі, паролі з великими і малими літерами, спеціальні символи).
2. Пояснення критеріїв міцності паролів (довжина паролю, складність символів, випадковість).
3. Розробка програми для генерації паролів:

- створення програми, яка здатна генерувати паролі різної довжини та складності;
- програма повинна мати можливість налаштовувати параметри генерації, такі як довжина, використання різних типів символів (літери верхнього та нижнього регістрів, цифри, спеціальні символи) та врахування вимог до міцності паролів. Тобто записувати введений пароль у файл;
- перед збереженням у файл паролі повинні хешуватися будь-яким алгоритмом хешування на власний вибір (наприклад MD5).

4. Згенерувати достатню кількість паролів у файл для їх аналізу (зберегти їх хеш у файл).
5. Підключити алгоритми хешування, наприклад ***MD5***. Для цього встановіть бібліотеку ***OpenSSl***:
   Для встановлення бібліотеки ***OpenSSl*** на комп'ютер дотримуйтеся наступних інструкцій (https://chocolatey.org/install#individual):

- відкрийте вікно ***PowerShell*** для ***Windows*** через права адміністратора (для цього натисніть комбінацію клавіш «***Windows + R»*** у вікні напишіть ***PowerShell***, а потім натисніть комбінацію клавіш «***Ctrl + Shift + Enter»***, таким чином у вас відкриється ***Powershell*** з правами адміністратора)

- у вікні ***PowerShell*** напишіть наступні команди:
  Set-ExecutionPolicy AllSigned, має з'явитися наступне:

```
PS C:\WINDOWS\system32> Set-ExecutionPolicy AllSigned

Изменение политики выполнения
Политика выполнения защищает компьютер от ненадежных сценариев. Изменение политики выполнения может поставить под
угрозу безопасность системы, как описано в разделе справки, вызываемом командой about_Execution_Policies и
расположенном по адресу https:/go.microsoft.com/fwlink/?LinkID=135170 . Вы хотите изменить политику выполнения?        [Y] Да - Y  [A] Да для всех - A  [N] Н
ет - N  [L] Нет для всех - L  [S] Приостановить - S  [?] Справка                (значением по умолчанию является "N"):Y
```

  Або «***Set-ExecutionPolicy Bypass -Scope Process***»,

```
PS C:\WINDOWS\system32> Set-ExecutionPolicy Bypass -Scope Process

Изменение политики выполнения
Политика выполнения защищает компьютер от ненадежных сценариев. Изменение политики выполнения может поставить под
угрозу безопасность системы, как описано в разделе справки, вызываемом командой about_Execution_Policies и
расположенном по адресу https:/go.microsoft.com/fwlink/?LinkID=135170 . Вы хотите изменить политику выполнения?
[Y] Да - Y  [A] Да для всех - A  [N] Нет - N  [L] Нет для всех - L  [S] Приостановить - S  [?] Справка
(значением по умолчанию является "N"):Y
```

- потім встановіть засіб ***chocolatey*** (https://chocolatey.org/install#individual), який встановить Вам бібліотеку ***OpenSSl***
- перевірка встановлення: Після завершення встановлення ви можете перевірити, чи встановлено ***OpenSSL***, відкривши командний рядок або ***PowerShell*** і виконавши команду:
  «openssl version»

6. Підключення бібліотеки ***OpenSSL*** до проекту в ***CodeBlocks:***

- додайте шлях до заголовкових файлів:
  У властивостях вашого проекту перейдіть до розділу "***Search directories***" (пошук директорії). Додайте шлях до директорії ***include*** бібліотеки ***OpenSSL***, де знаходяться заголовні файли (наприклад, ***/usr/include/openssl***). Ви можете це зробити, вибравши "***Project" -> "Build options" -> "Search directories" -> "Compiler***" та натиснути кнопку **"Add"**.
- додайте шлях до бібліотек:
  Додайте шлях до директорії ***lib*** бібліотеки ***OpenSSL***, де знаходяться скомпільовані бібліотеки (наприклад, ***/usr/lib***). Ви можете це зробити, вибравши "***Project" -> "Build options" -> "Search directories" -> "Linker***" та натискаючи кнопку ***"Add"***.
- додайте бібліотеки до проекту:
  У властивостях вашого проекту перейдіть до розділу ***"Linker settings"*** (налаштування лінкера). Додайте ***-lssl*** та ***-lcrypto*** до списку бібліотек. Ви можете це зробити, вибравши ***"Project" -> "Build options" -> "Linker settings"*** та натискаючи кнопку ***"Add"*** для кожної бібліотеки.
  Після чого підключення бібліотеки виглядає так:

```
#include <openssl/md5.h>
```

Приклад коду з підключеною функцією хешування MD5:

```
// Function to calculate MD5 hash of a string
string calculateMD5(const string& input) {
    unsigned char digest[MD5_DIGEST_LENGTH];
```

```
    MD5_CTX context;
    MD5_Init(&context);
    MD5_Update(&context, input.c_str(), input.length());
    MD5_Final(digest, &context);
stringstream ss;
    for(int i = 0; i < MD5_DIGEST_LENGTH; ++i) {
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    }
    return ss.str();
}
```

7. Встановлення програми ***John the Ripper***.

- підготуйте файл з паролями: Переконайтеся, що файл з паролями (наприклад ***passwords.txt***) правильно відформатований, з одним паролем на кожному рядку. Якщо у вас є кілька паролів у файлі, то ***John the Ripper*** спробує зламати кожен з них.
- Запустіть ***John the Ripper***: Відкрийте термінал або командний рядок та перейдіть до каталогу, де встановлений ***John the Ripper***.
- Запустіть ***John the Ripper*** з файлом паролів: Використовуйте команду ***john***, за якою слідує назва файлу з паролями, який ви хочете зламати. Наприклад:

```
john passwords.txt
```

- чекайте аналізу: John the Ripper почне аналізувати файл паролів та спробувати зламати паролі, використовуючи різні техніки, такі як словники, брутфорс атаки і т.д.

  У ***John the Ripper*** аналіз файлу зазвичай відображається у вигляді зламаних паролів та відповідних хешів, які були розкриті. Результат може виглядати приблизно так:

```
Loaded 1 password hash (MD5 [MD5 128/128 XOP 4x1])
admin              (admin)
guesses: 1  time: 0:00:00:01 DONE (Mon Jan 10 12:34:56 2022)  c/s:
32767  trying: 123456 - admin
Use the "--show" option to display all of the cracked passwords
reliably
```

У цьому прикладі:

"admin" - це зламаний пароль,

"(admin)" - відповідний хеш,

"guesses" - кількість спроб, які знадобилися для злому пароля,

"time" - час, який знадобився для злому пароля,

"c/s" - швидкість перевірки паролів (відповідно до кількості спроб на секунду).

"trying" - діапазон значень, який використовувався для спроб злому пароля.

Також можна скористатися опцією ***--show***, щоб відобразити всі зламані паролі з відповідними хешами.

- Ознайомтеся з результатами: Як тільки ***John the Ripper*** завершить свій аналіз, він відобразить зламані паролі, якщо такі є разом із відповідними хешами та часом, необхідним для їх зламу.
- Огляньте вихідний файл: ***John the Ripper*** може згенерувати вихідний файл, що містить зламані паролі та іншу інформацію. Ви можете оглянути цей файл для подальшого аналізу.

Один з можливих варіантів програми для генерації паролів за допомогою вбудованої хеш функції та збереження паролів у файл:

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include <openssl/md5.h>

using namespace std;

// Function to calculate MD5 hash of a string
string calculateMD5(const string& input) {
    unsigned char digest[MD5_DIGEST_LENGTH];
    MD5_CTX context;
    MD5_Init(&context);
    MD5_Update(&context, input.c_str(), input.length());
    MD5_Final(digest, &context);

    stringstream ss;
    for(int i = 0; i < MD5_DIGEST_LENGTH; ++i) {
        ss << hex << setw(2) << setfill('0') << (int)digest[i];
    }
    return ss.str();
}

// Function to append the password and its hash to a file
void appendPasswordToFile(const string& password, const string& filename)
{
    ofstream outfile(filename, ios::app); // Open the file for appending
    if (outfile.is_open()) {
        string hashedPassword = calculateMD5(password); // Calculate MD5
hash of the password
        outfile << password << " - " << hashedPassword << endl; // Write
the password and its hash to the file
        cout << "Password and its hash have been appended to the file: "
<< filename << endl;
        outfile.close(); // Close the file
    } else {
        cout << "Error appending password and its hash to the file: " <<
filename << endl;
    }
}
```

```
int main() {
    string password;
    string filename = "passwords.txt"; // Name of the file to store
passwords

    cout << "Enter the password: ";
    getline(cin, password); // Read the password from console

    appendPasswordToFile(password, filename); // Append the password and
its hash to the file

    return 0;
}
```